Space Mission Analysis, Winter Term 2026/27

Assignment 1 – due 17.11.2026 17:59

The assignment shall be done in groups of 2-3 students. However, each student must upload the code and report individually via ISIS!

Reports must include the plots, explanations and a detailed analysis in PDF format as well as your MATLAB code.

Alternatively, you can submit a MATLAB live script (.mlx) showing and commenting all plots and calculations instead of the PDF.

Tasks (34p.):

Assume the following ω = 0°, Ω_0 = 0°, υ_0 = 0°. You can select t_0 yourself. Make sure that the code is executable when you hand it in and that your computed values are consistent.

1. Kepler orbit propagation **(10p.)**
   a. Implement a function that propagates a Keplerian undisturbed orbit over time. Thereby use the Newton method and choose a reasonable time step. **(5p.)**
   b. Calculate the inclination i of a sun-synchronous orbit (SSO) with a perigee height of 550 km and an eccentricity e = 0.01, propagate it for 30 orbit revolutions and save position vectors in cartesian coordinates (ICRF). **(1p.)**
   c. Plot the orbit in a 3D plot and the ground track of the satellite. **(1p.)**
   d. Adapt your Kepler propagation function to include node drift, i.e. $\dot{\Omega} \neq 0$, caused by the J2 perturbation. **(1p.)**
   e. Propagate the SSO of task 1 with the J2 perturbation for 30 orbit revolutions and save position vectors in cartesian coordinates (ICRF). **(1p.)**
   f. Plot the orbit in a 3D plot and the ground track of the satellite. **(1p.)**

2. Equation of Motion propagation **(10p.)**
   a. Implement a Runge-Kutta solver that is able to use different RK methods by providing their coefficients. It should be capable of: **(8p.)**
      i. Solving different ODEs provided to the solver via function handle
      ii. Using methods of different orders
      iii. Step size control for embedded methods with the lower order used for error estimation

      Using a class-based approach is recommended.
   b. Propagate the unperturbed EoM for a SSO with a perigee height of 550 km and an eccentricity e = 0.01 for 30 orbit revolutions and save position vectors in cartesian coordinates (ICRF). **(1p.)**
   c. Propagate the perturbed EoM with the J2 perturbation for the SSO for 30 orbit revolutions and save position vectors in cartesian coordinates (ICRF). **(1p.)**

3. Comparison of propagation methods **(14p.)**
   Compare the perturbed Kepler propagation, the Runge-Kutta-Fehlberg 4(5) and the Runge-Kutta-Fehlberg 7(8) methods using your solvers.
   For this, propagate a SSO orbit with a height of 650 km and an eccentricity of e =

0.01 and a Geostationary Transfer Orbit with a periapsis height of 650 km for 36 hours. In your analysis include at least:

i. Execution time comparison
ii. Step size analysis including rejected step sizes
iii. Visualization of differences in orbit states for both orbits between methods
iv. Direct comparison of all three methods

Your report should include all graphs, captions and explanations of what you did. The report's form, structure, language etc. will be accounted for with **3p.** as well as the code's structure, appropriate comments etc. will be accounted for with **3p.**

I strongly recommend to write your script as lean as possible and to try to avoid repeating code. Use loops and parameter sets to calculate or plot different orbits.

All calculations and simulations must be done using MATLAB with own functions (no usage of libraries or proprietary or open-source functions)!

Exceptions to this are the functions *eci2lla, lla2ecef, eci2ecef* and their reverse versions. Furthermore, functions like *wrapTo360, wrapTo2Pi* and similar as well as *geoplot* are exceptions.

ode45 and all other *odeXX* functions are prohibited specifically, you have to implement the Runge-Kutta-solver yourself.